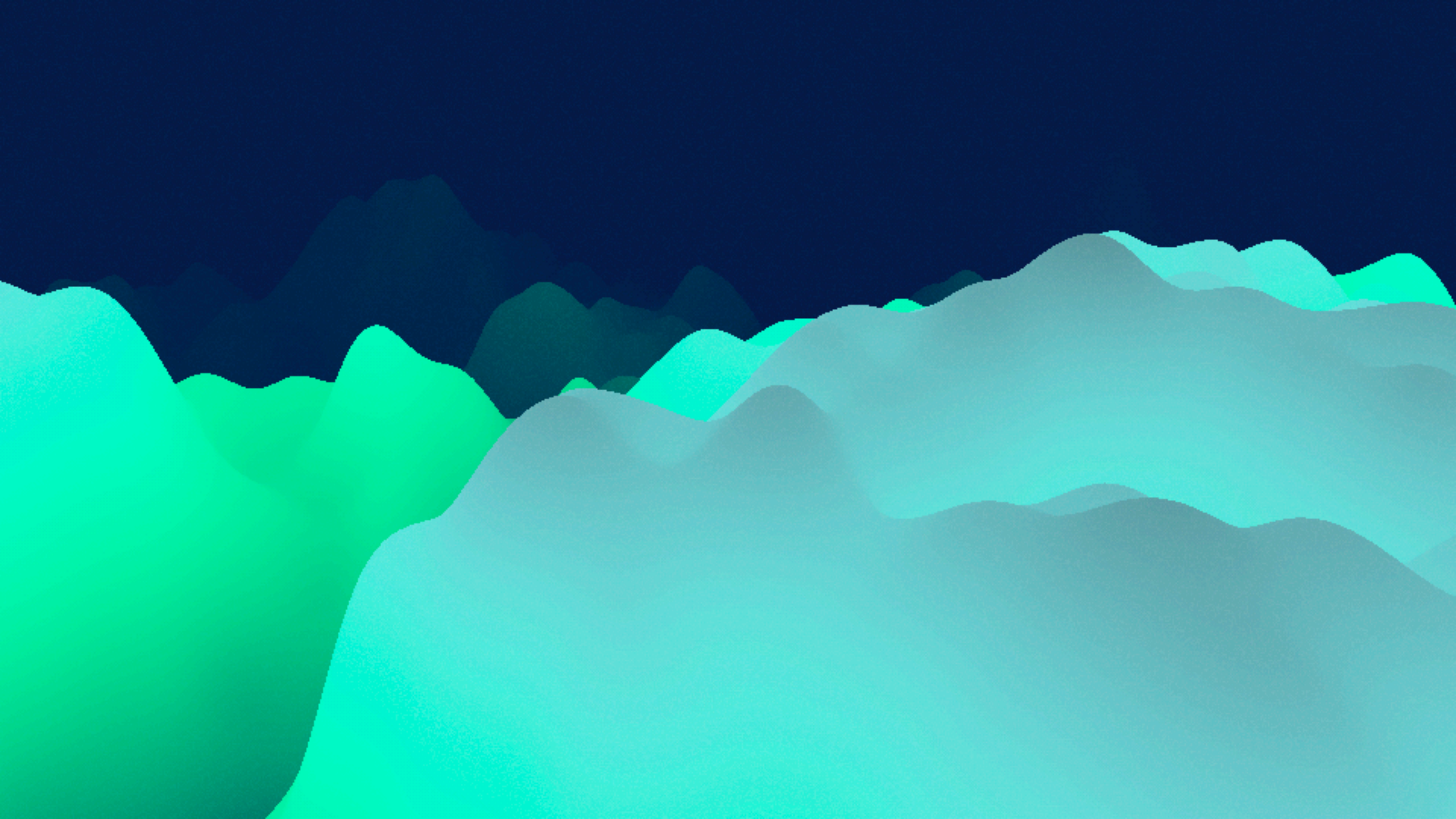






考点1 线性表的基本操作

基本操作	顺序表	链表	静态链表
按位查找	$O(1)$ 顺序表支持随机访问	$O(n)$ 在链表查找第 <i>i</i> 个元素只能遍历	$O(n)$ 在链表查找第 <i>i</i> 个元素只能遍历
按值查找	$O(n)$ 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功	$O(n)$ 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功	$O(n)$ 需要一个个遍历元素 并与查找元素值对比 才能确定是否查找成功
插入	$O(n)$ 需要大量移动元素腾出空位 如果剩余连续空间不够 还需将整个顺序表复制粘贴到 一片新的更大的连续空间	$O(n)+O(1)$ 需要遍历来确定插入位置的前驱节点 然后修改指针进行插入	$O(n)+O(1)$ 需要遍历来确定插入位置的前驱节点 然后修改数组进行插入
删除	$O(n)$ 需要大量移动元素覆盖被删元素	$O(n)+O(1)$ 需要遍历来确定删除位置的前驱节点 然后修改指针进行删除	$O(n)+O(1)$ 需要遍历来确定删除位置的前驱节点 然后修改数组进行删除



考点1 线性表的基本操作

【2023】在下列对顺序存储的有序表（长度为 n ）实现给定操作的算法中，平均时间复杂度为 $O(1)$ 的是

- A. 查找包含指定值元素的算法
- B. 插入包含指定值元素的算法
- C. 删除第 i ($1 \leq i \leq n$) 个元素的算法
- D. 获取第 i ($1 \leq i \leq n$) 个元素的算法**



考点1 线性表的基本操作

【2016】已知一个带有表头结点的循环双链表L，节点结构为

prev	data	next
------	------	------

，其中prev和next分别是指向其直接前驱和直接后继结点的指针。现要删除指针p所指的节点，正确的语句序列是

- A. `p -> next -> prev = p -> prev; p -> prev -> next = p -> prev; free(p);`
- B. `p -> next -> prev = p -> next; p -> prev -> next = p -> next; free(p);`
- C. `p -> next -> prev = p -> next; p -> prev -> next = p -> prev; free(p);`
- D. `p -> next -> prev = p -> prev; p -> prev -> next = p -> next; free(p);`**



考点1 线性表的基本操作

【2021】已知头指针h指向一个带头节点的非空循环单链表，结点结构为

data	next
------	------

，其中next是指向直接后继

结点的指针，p是尾指针，q是临时指针。现要删除该链表的第一个元素，正确的语句序列是

A. `h -> next = h -> next -> next; q = h -> next; free(q);`

B. `q = h -> next; h -> next = h -> next -> next; free(q);`

C. `q = h -> next; h -> next = q -> next; if(p != q) p = h; free(q);`

D. `q = h -> next; h -> next = q -> next; if(p == q) p = h; free(q);`



考点1 线性表的基本操作

习题训练

1. 如果线性表的最常用操作是“频繁访问第 i 个元素”，且对访问速度要求较高，应使用的存储结构是

A. 单链表

B. 顺序表

C. 双向链表

D. 循环链表



考点1 线性表的基本操作

习题训练

2. 关于顺序表和链表在执行插入、删除操作时的空间与效率表现，下列说法正确的是
- A. 顺序表的插入、删除操作时间复杂度低，且无需移动元素
 - B. 链表插入删除效率高，但由于每个节点需额外存储指针，空间利用率低**
 - C. 顺序表在任意位置插入或删除元素的平均时间复杂度为 $O(1)$
 - D. 链表访问第 i 个节点的时间复杂度为 $O(1)$



考点1 线性表的基本操作

习题训练

3. 在一个带头结点的循环双向链表中，指针p指向某个中间结点。现要删除p的前驱节点（即p->prev指向的结点），下列语句顺序正确的是

A. p->prev = p->prev->prev; free(p->prev); p->prev->next = p;

B. p->prev->prev->next = p; q = p->prev->prev; free(p->prev); p->prev = q;

C. p->prev->next = p; free(p->prev); p->prev = p->prev->prev;

D. free(p->prev); p->prev = p->prev->prev; p->prev->next = p;

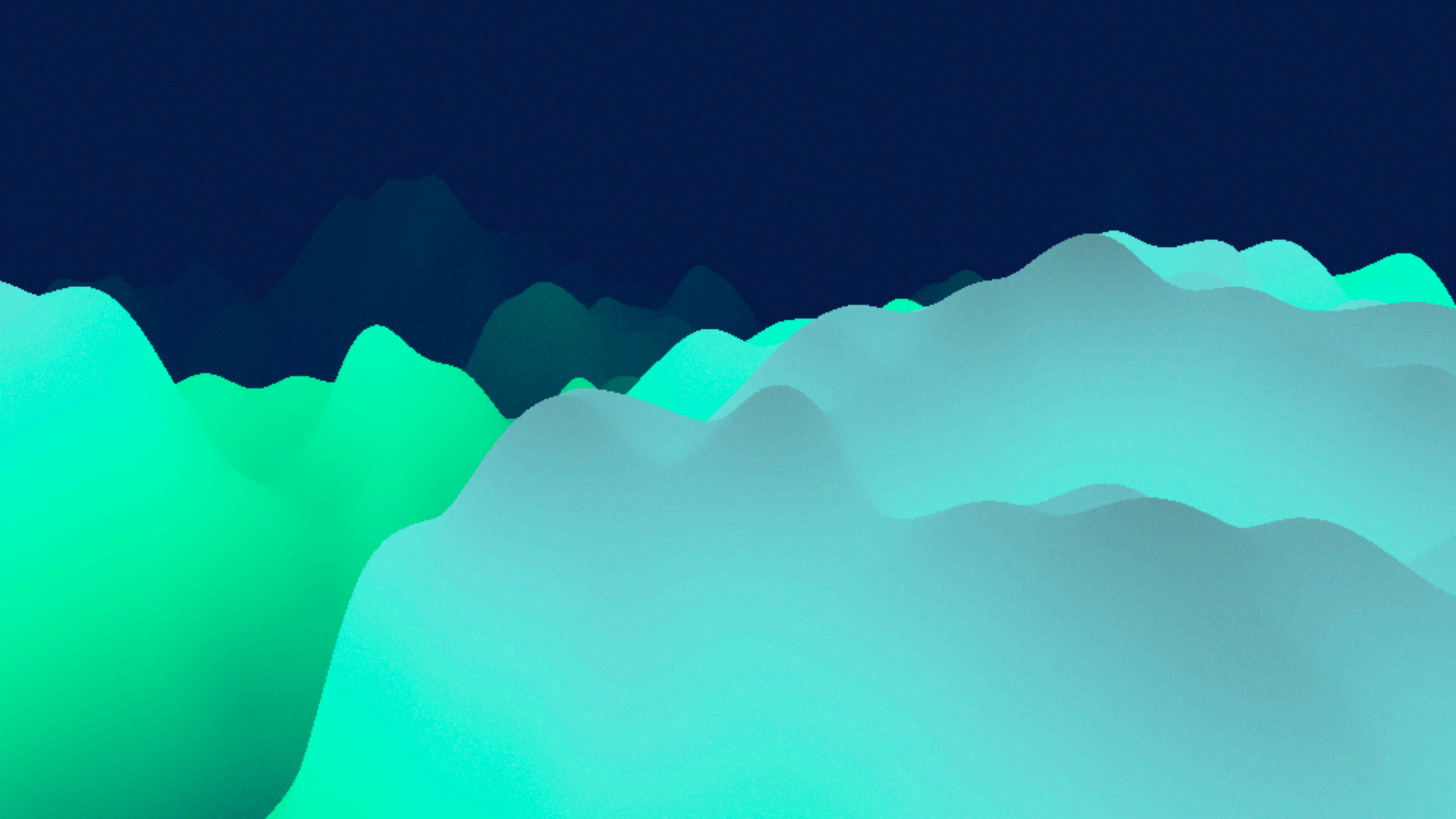


考点1 线性表的基本操作

习题训练

4. 已知带头结点的循环单链表，其头指针为h，链表结构为{data, next}，其中next为指向后继的指针。现有指针q是一个新建节点，执行以下语句：`q->next = h->next; h->next = q;`则该操作的结果是

- A. 将结点 q 插入到链表末尾
- B. 将结点 q 插入到头结点之后，成为新的首元结点**
- C. 删除原来的第一个有效节点
- D. 将结点 q 插入到链表末尾，并将其变为新的头结点





考点2 栈和队列的基本操作

【2020】对空栈S进行Push和Pop操作，入栈序列为a, b, c, d, e, 经过Push、Push、Pop、Push、Pop、Push、Push、Pop操作后得到的出栈序列是

A. b, a, c

B. b, a, e

C. b, c, a

D. b, c, e



考点2 栈和队列的基本操作

【2011】元素a, b, c, d, e依次进入初始为空的栈中，若元素入栈后可停留、可出栈，直到所有元素都出栈，则在所有可能的出栈序列中，以元素d开头的序列个数是

A. 3

B. 4

C. 5

D. 6



考点2 栈和队列的基本操作

【2022】给定有限符号集 S ， in 和 out 均为 S 中所有元素的任意排列。对于初始为空的栈 ST ，下列叙述中，正确的是

- A. 若 in 是 ST 的入栈序列，则不能判断 out 是否为其可能的出栈序列
- B. 若 out 是 ST 的出栈序列，则不能判断 in 是否为其可能的入栈序列
- C. 若 in 是 ST 的入栈序列， out 是对应 in 的出栈序列，则 in 和 out 一定不同
- D. 若 in 是 ST 的入栈序列， out 是对应 in 的出栈序列，则 in 和 out 可能互为倒序**



考点2 栈和队列的基本操作

【2010】某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素a，b，c，d，e依次入此队列后再进行出队操作，则不可能得到的出队序列是

A. b, a, c, d, e

B. d, b, a, c, e

C. d, b, c, a, e

D. e, c, b, a, d



考点2 栈和队列的基本操作

【2018】 现有队列Q与栈s，初始时Q中的元素依次是1，2，3，4，5，6（1在队头），s为空。若仅允许下列3种操作：①出队并输出出队元素；②出队并将出队元素入栈；③出栈并输出出栈元素，则不能得到的输出序列是

A. 1，2，5，6，4，3

B. 2，3，4，5，6，1

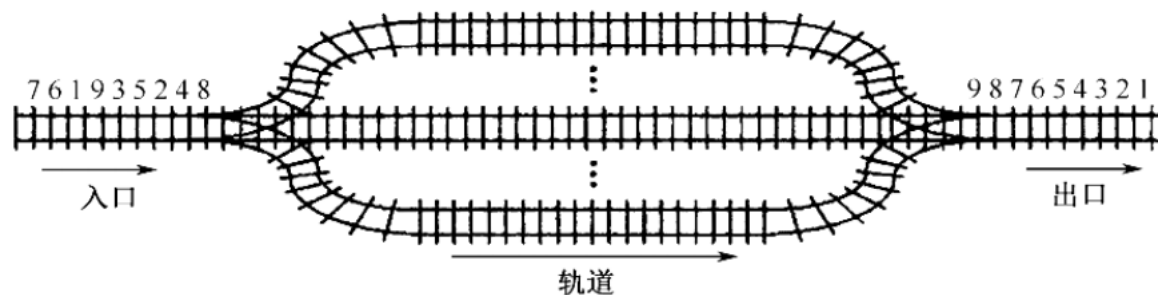
C. 3，4，5，6，1，2

D. 6，5，4，3，2，1



考点2 栈和队列的基本操作

【2016】设有如下图所示的火车车轨，入口和出口之间有 n 条轨道，列车的行进方向均为从左至右，列车可驶入任意一条轨道。现有编号为1~9的9列列车，驶入的次序依次是8，4，2，5，3，9，1，6，7。若期望驶出的次序依次为1~9，则 n 至少是



- A. 2
- B. 3
- C. 4**
- D. 5



考点2 栈和队列的基本操作

习题训练

5. 链栈（不带头结点）执行push操作，向栈中压入节点x，应执行

A. `top = x; x -> next = top;`

B. `x -> next = top; top = x;`

C. `x -> next = NULL; top = x;`

D. `top -> next = x; x = top;`



考点2 栈和队列的基本操作

习题训练

6. 若元素 a, b, c, d, e, f 依次入栈，允许入栈、出栈操作交替进行，但不允许连续4次出栈操作，则以下不可能得到的出栈序列是

A. $acbedf$

B. $abcdef$

C. $fedcba$

D. $abcdfe$



考点2 栈和队列的基本操作

习题训练

7. 栈的入栈序列为 $1, 2, 3, \dots, n$ ，出栈序列为 $P_1, P_2, P_3, \dots, P_n$ 。若 $P_3=3$ ，则 P_4 的可能取值的个数是

A. $n-3$

B. $n-2$

C. $n-1$

D. 无法确定



考点2 栈和队列的基本操作

习题训练

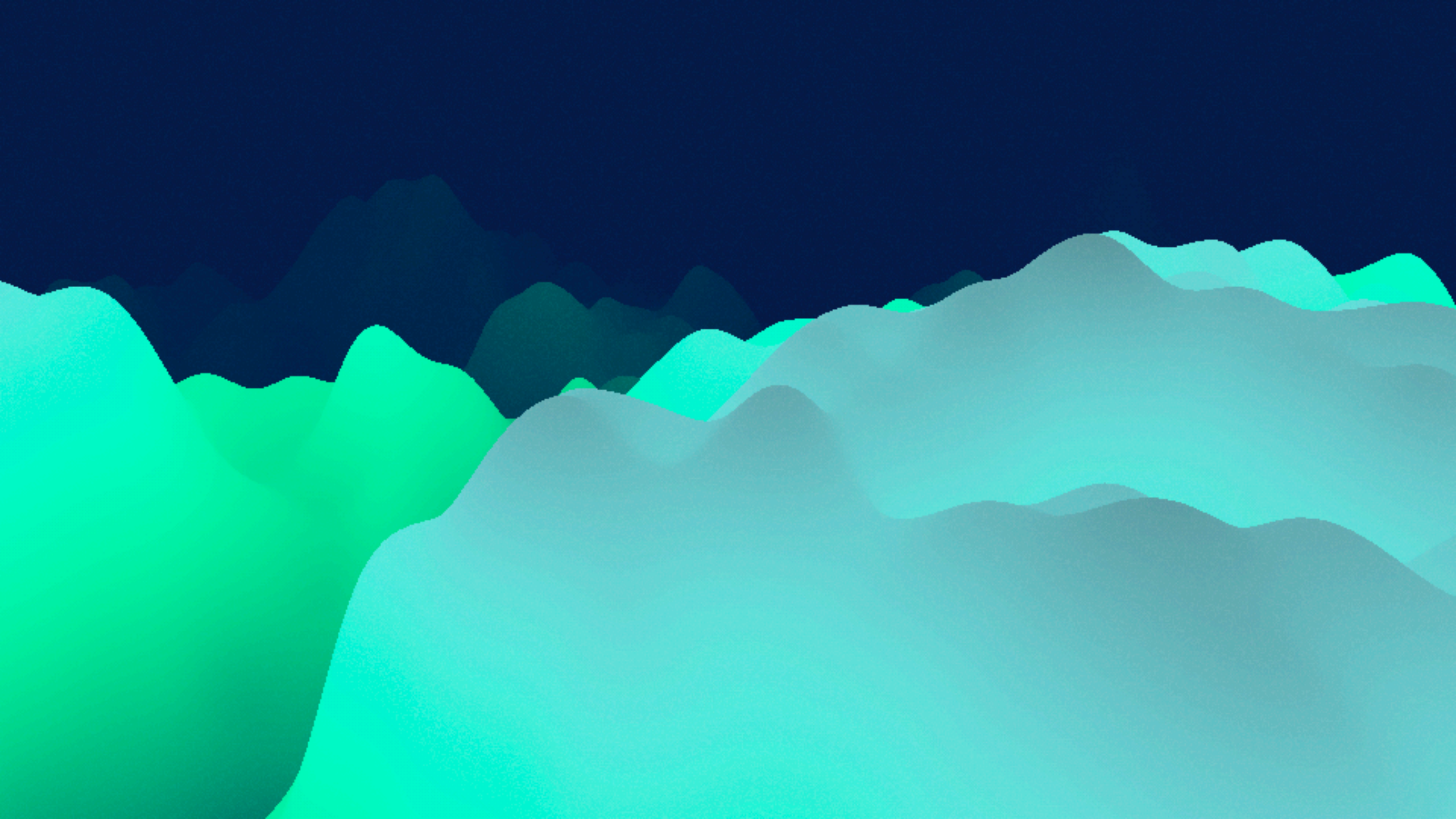
8. 某数据结构为“双端队列”，其特性为只能从一端入队，允许从两端出队。若元素 a, b, c, d 依次入队，允许入队和出队操作交替进行，则下列不可能的出队序列是

A. d, c, b, a

B. d, b, c, a

C. a, b, c, d

D. a, c, d, b





考点3 卡特兰数

$$C_n = \frac{1}{n+1} C_{2n}^n$$

1. n个元素进行栈操作（入栈序列确定）有多少种**合法出栈序列**
2. 先序序列含n个不同元素时（先序序列确定），能够确定多少种**不同的二叉树**



考点3 卡特兰数

【2015】先序序列为a, b, c, d的不同二叉树的个数是

A. 13

B. 14

C. 15

D. 16

$$C_n = \frac{1}{n+1} C_{2n}^n$$

1. n个元素进行栈操作（入栈序列确定）有多少种**合法出栈序列**

2. 先序序列含n个不同元素时（先序序列确定），能够确定多少种**不同的二叉树**



考点3 卡特兰数

习题训练

9. 元素 a, b, c, d, e, f 依次进入一个初始为空的栈中，若每个元素入栈后可停留、可出栈，直到所有元素都出栈，则所有可能的出栈序列总数是

A. 20

B. 42

C. 132

D. 720

习题训练

10. 由3个结点可以构造出的不同二叉树数量是

A. 2

B. 3

C. 4

D. 5

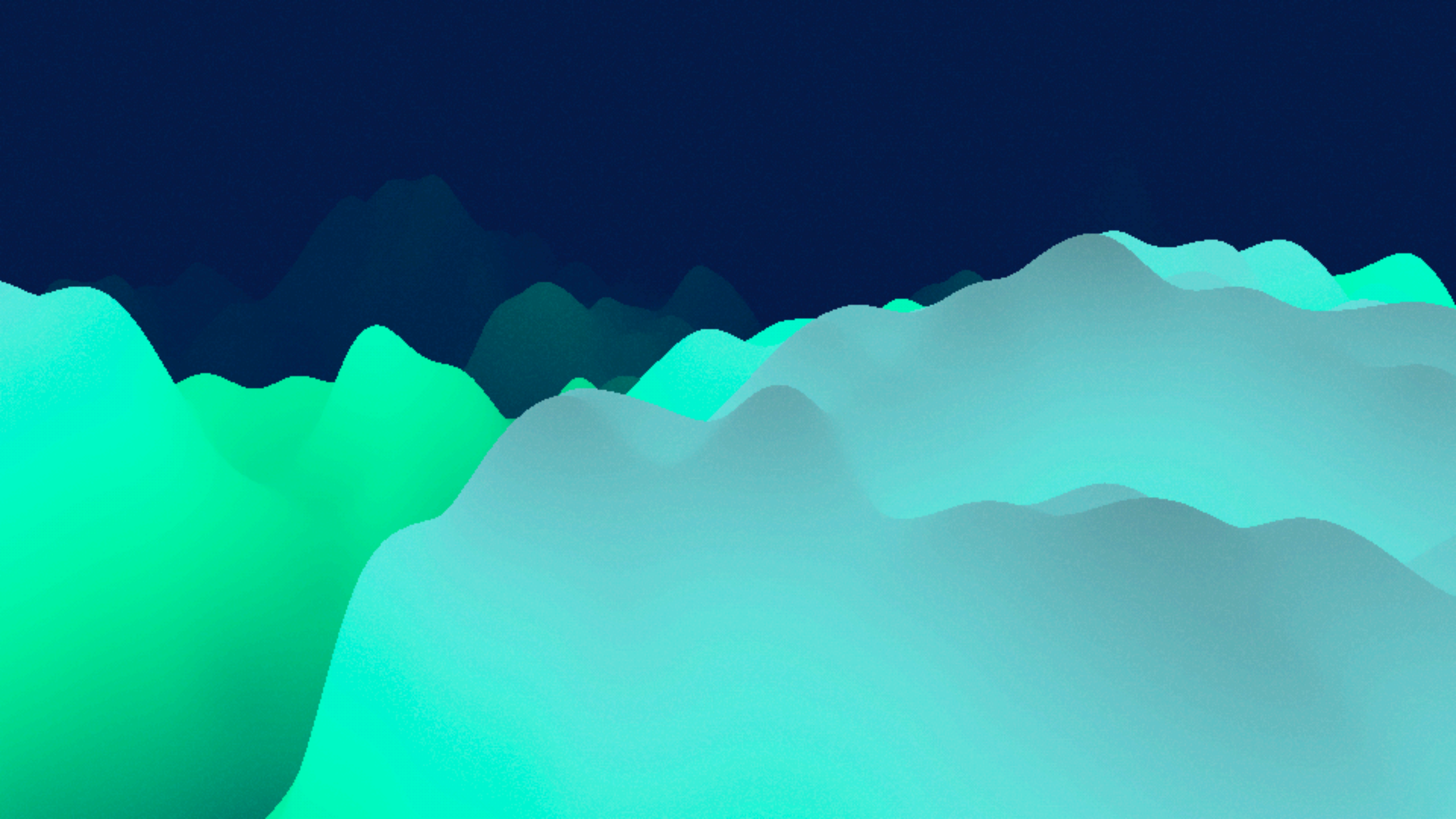


考点3 卡特兰数

习题训练

11. 具有5个结点的二叉树有着相同的先序和中序遍历序列“ooops”，则这样的二叉树可以确定的不同形状有多少？

- A. 1
- B. 3
- C. 5**
- D. 6

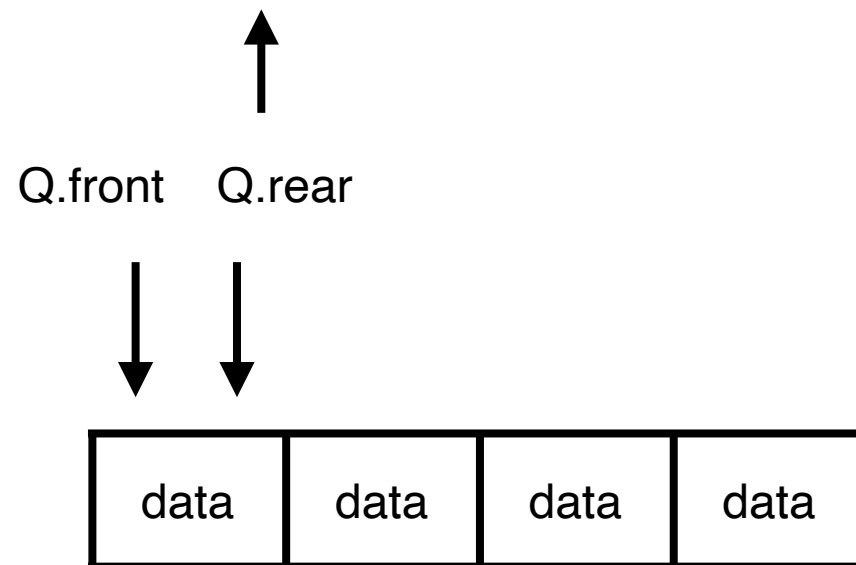




考点4 假溢出和循环队列

①队列的假溢出问题

依次操作:入队, 入队, 入队, 入队, 出队, 出队



此时队列非空, 但也无法再入队



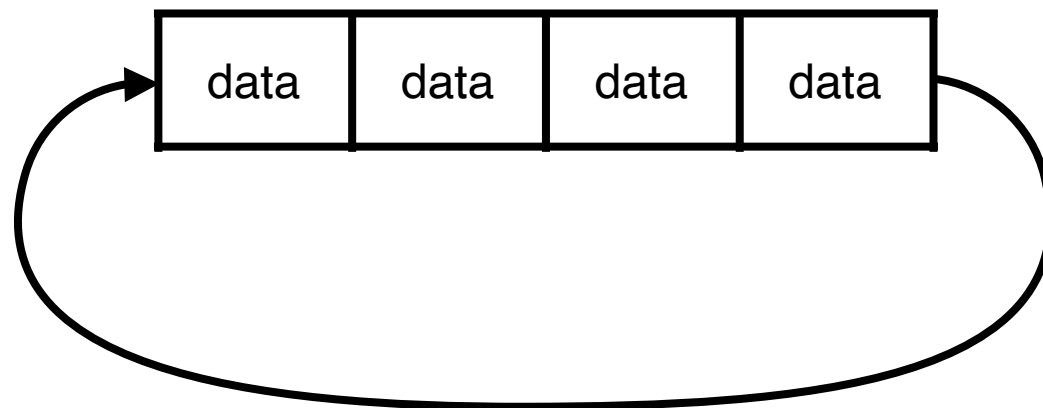
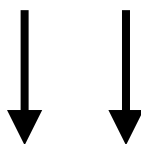
考点4 假溢出和循环队列

①队列的假溢出问题

依次操作：入队，入队，入队，入队，出队，出队



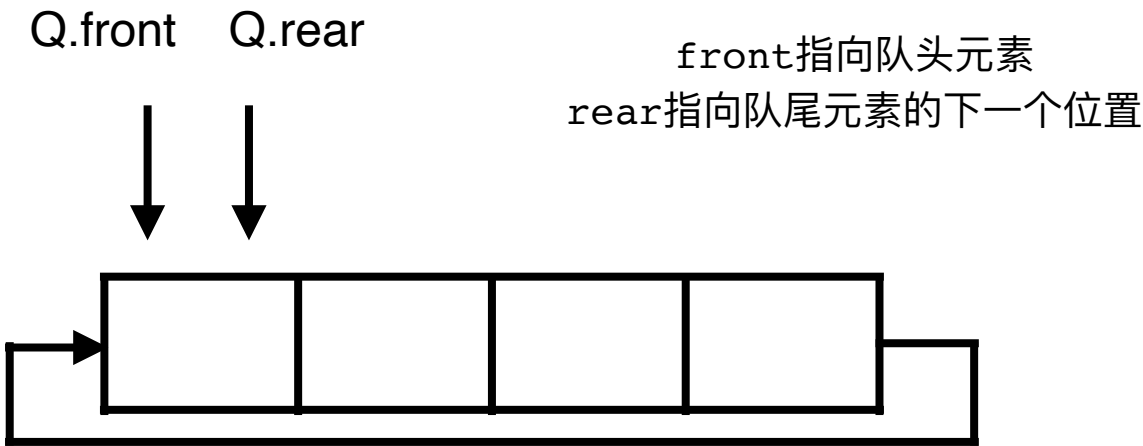
Q.front Q.rear



引入循环队列

考点4 假溢出和循环队列

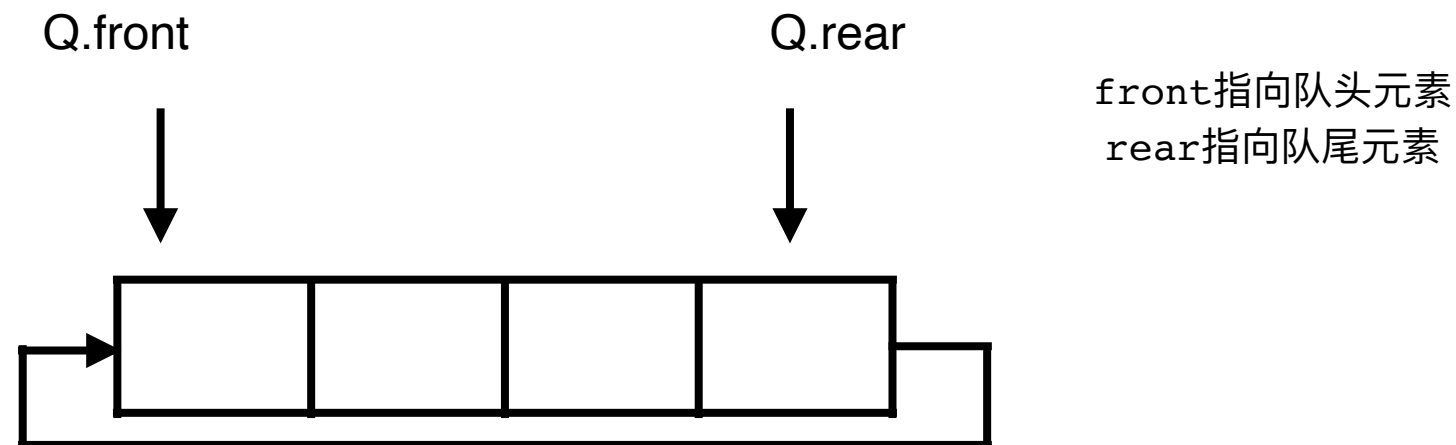
②循环队列的指针设置



队空	<code>rear == front (= 0)</code>
队满	<code>rear == front (= 0)</code>
入队	<code>Q.data[rear] = x;</code> <code>rear = (rear + 1) % MAXSIZE;</code>
出队	<code>x = Q.data[front];</code> <code>front = (front + 1) % MAXSIZE;</code>

考点4 假溢出和循环队列

②循环队列的指针设置



队空	$(rear+1) \% MAXSIZE == front$
队满	$(rear+1) \% MAXSIZE == front$
入队	$rear = (rear + 1) \% MAXSIZE;$ $Q[rear] = x;$
出队	$x = Q[front];$ $front = (front + 1) \% MAXSIZE;$

考点4 假溢出和循环队列

②循环队列的指针设置

队空	<code>rear == front (= 0)</code>
队满	<code>rear == front (= 0)</code>
入队	<code>Q.data[rear] = x;</code> <code>rear = (rear + 1) % MAXSIZE;</code>
出队	<code>x = Q.data[front];</code> <code>front = (front + 1) % MAXSIZE;</code>

front指向队头元素
rear指向队尾元素的下一个位置

队空	<code>(rear+1) % MAXSIZE == front</code>
队满	<code>(rear+1) % MAXSIZE == front</code>
入队	<code>rear = (rear + 1) % MAXSIZE;</code> <code>Q[rear] = x;</code>
出队	<code>x = Q[front];</code> <code>front = (front + 1) % MAXSIZE;</code>

front指向队头元素
rear指向队尾元素

考点4 假溢出和循环队列

③区别队空队满的三个方案

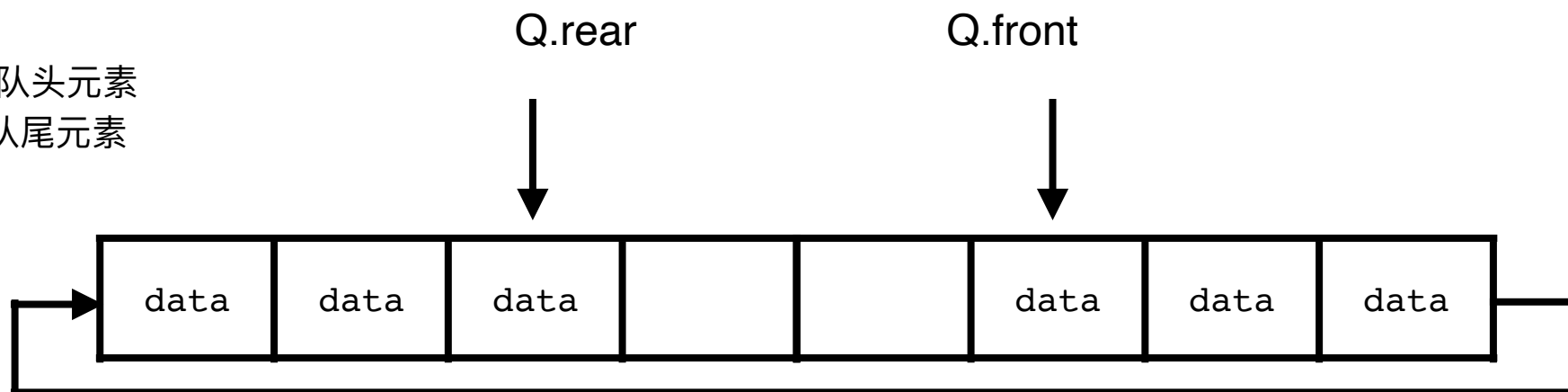
方案1	牺牲一个单元不放元素
方案2	设置tag，删除成功后置tag=0，插入成功后置tag=1
方案3	设置num表示队列中的元素个数



考点4 假溢出和循环队列

④循环队列的长度求解

front指向队头元素
rear指向队尾元素



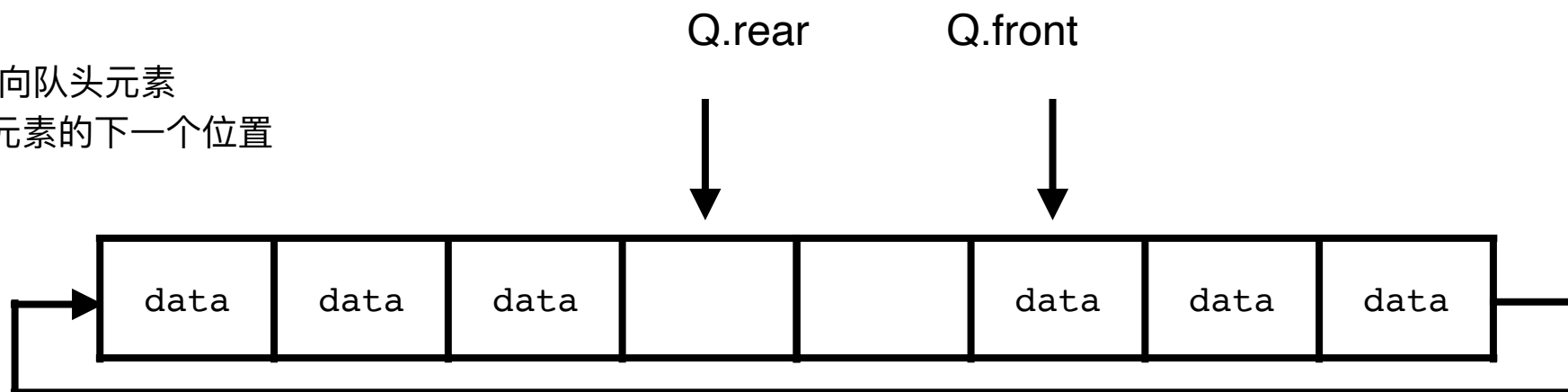
$$\text{循环队列长度} = (\text{MAXSIZE} + \text{rear} - \text{front} + 1) \% \text{MAXSIZE}$$



考点4 假溢出和循环队列

④循环队列的长度求解

front指向队头元素
rear指向队尾元素的下一个位置



$$\text{循环队列长度} = (\text{rear} - \text{front} + \text{MAXSIZE}) \% \text{MAXSIZE}$$



考点4 假溢出和循环队列

【2011】 已知循环队列存储在一维数组 $A[0 \dots n-1]$ 中，且队列非空时 $front$ 和 $rear$ 分别指向队首元素和队尾元素。若初始时队列为空，且要求第一个进入队列的元素存储在 $A[0]$ 处，则初始时 $front$ 和 $rear$ 的值分别是

A. 0, 0

B. 0, $n-1$

C. $n-1$, 0

D. $n-1$, $n-1$



考点4 假溢出和循环队列

【2014】循环队列放在一维数组 $A[0 \dots M-1]$ 中， $end1$ 指向队首元素， $end2$ 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 $M-1$ 个元素。初始时空。下列判断队空和队满的条件中，正确的是

A. 队空： $end1 == end2$;

B. 队空： $end1 == end2$;

C. 队空： $end2 == (end1 + 1) \bmod M$;

D. 队空： $end1 == (end2 + 1) \bmod M$;

队满： $end1 == (end2 + 1) \bmod M$

队满： $end2 == (end1 + 1) \bmod (M-1)$

队满： $end1 == (end2 + 1) \bmod M$

队满： $end2 == (end1 + 1) \bmod (M-1)$



考点4 假溢出和循环队列

【2019】请设计一个队列，要求满足：①初始时队列为空；②入队时，允许增加队列占用空间；③出队后，出队元素所占用的空间可重复使用，即整个队列所占用的空间只增不减；④入队操作和出队操作的时间复杂度始终保持为 $O(1)$ 。请回答：

- 1) 该队列是应选择链式存储结构，还是应选择顺序存储结构？
- 2) 画出队列的初始状态，并给出判断队空和队满的条件。
- 3) 画出第一个元素入队后的队列状态。
- 4) 给出入队操作和出队操作的基本过程。



考点4 假溢出和循环队列

习题训练

12. 一个循环队列， f 为当前队列头元素的前一位置， r 为队尾元素的位置，假定队列中元素的个数小于 n ，计算队列中元素个数的公式为

A. $r - f$

B. $(n + f - r) \% n$

C. $n + r - f$

D. $(n + r - f) \% n$



考点4 假溢出和循环队列

习题训练

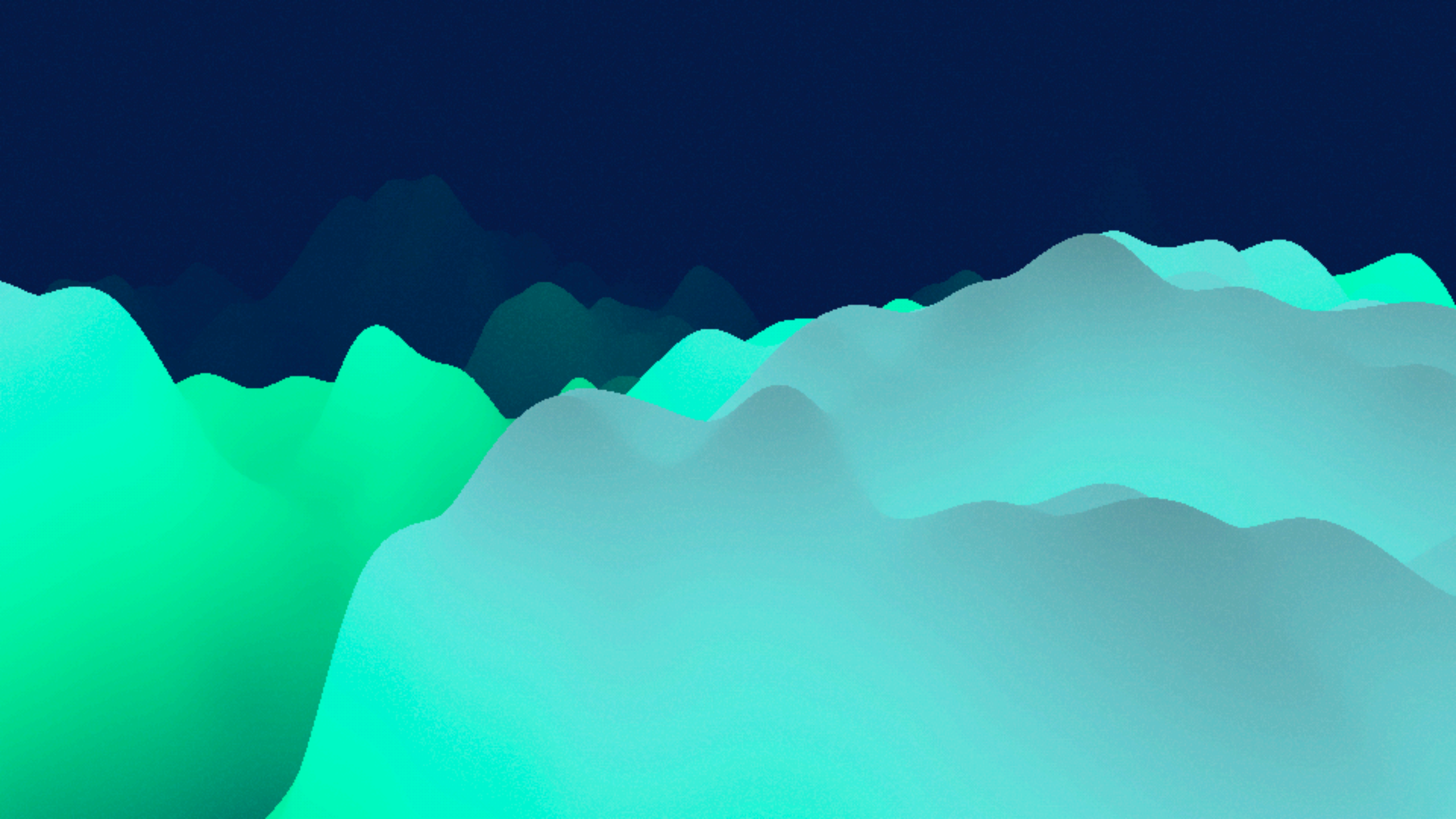
13. 现有一循环队列，使用长度为 n 的数组实现，入队操作为： $A[\text{rear}] = x; \text{rear} = (\text{rear} + 1) \% n$ ；出队操作为： $\text{front} = (\text{front} + 1) \% n$ ；队列保留一个空位区分队空队满，在这个设计下，判断“队列为空”的条件应为

A. $\text{rear} == \text{front}$

B. $(\text{rear} + 1) \% n == \text{front}$

C. $(\text{front} + 1) \% n == \text{rear}$

D. $\text{rear} == 1 \ \&\& \ \text{front} == -1$





考点5 栈的应用

①括号匹配

括号匹配需要解决的问题： 1.左括号必须用相同类型的右括号闭合

2.左括号必须以正确的顺序闭合

3.每个右括号都有一个对应的相同类型的左括号

每当出现一种左括号，计算机便等待对应的右括号
但在此期间若出现了另一种左括号，则将其视为更高优先级，转而等待新类型的右括号
若遇到当前等待的右括号，则成对消除



考点5 栈的应用

①括号匹配

【2025】对于括号匹配问题，符号栈初始为空，容量为3，不能实现的表达式是

- A. $(a+[b+(c+d)/e]+f)+g-h$
- B. $[a*((b+c)/(d-e)+f/g)]-h$
- C. $[a*(b-(c-d)*e/(f+g))-h]$
- D. $[a-(b+[c*(d+e)-f]+g+h)]$**

表达式扫描结束但操作符栈非空时
依次弹出操作符栈中的符号并计算

考点5 栈的应用

②表达式求值

中缀表达式求值	后缀表达式求值	前缀表达式求值
从左到右扫描		从右到左扫描
扫描到操作数 直接入操作数栈		
扫描到操作符 依次弹出栈中优先级 大于等于 当前运算符的所有操作，碰到左括号“(“ 或栈空停止 每出栈一个操作符，将操作数栈栈顶的两个元素取出进行计算 并将结果压入操作数栈 最后将当前操作符压入操作符栈	扫描到操作符，即将栈顶的两个元素取出进行计算，并将结果压栈	
第一个出栈的操作数放在运算符的右边		第一个出栈的操作数放在运算符的左边
扫描到左括号“(“，入栈 扫描到右括号“)“ 依次弹出栈中运算符并输出 直到左括号“(“弹出为止	前/后缀表达式中没有括号	

考点5 栈的应用

③中缀表达式转后缀表达式（最常考）

扫描顺序	从左到右扫描
遇到操作数	直接输出，不做栈操作
遇到运算符	依次弹出栈中优先级大于等于当前运算符的所有运算符并输出 碰到左括号“（”或栈空停止 最后入栈当前扫描的运算符
遇到界限符	遇到左括号“（”，入栈 遇到右括号“）”，依次弹出栈中运算符并输出，直到左括号“（”弹出为止

考点5 栈的应用

③中缀表达式转后缀表达式（最常考）

【2012】已知操作符包括“+”“-”“*”“/”“(”和“)””。将中缀表达式 $a+b-a*((c+d)/e-f)+g$ 转换为等价的后缀表达式 $ab+acd+e/f-* -g+$ 时，用栈来存放暂时还不能确定运算次序的操作符。栈初始时空，转换过程中同时保存在栈中的操作符的最大个数是

- A. 5
- B. 7
- C. 8
- D. 11

扫描顺序	从左到右扫描
遇到操作数	直接输出，不做栈操作
遇到运算符	依次弹出栈中优先级大于等于当前运算符的所有运算符并输出 碰到左括号“(”或栈空停止 最后入栈当前扫描的运算符
遇到界限符	遇到左括号“(”，入栈 遇到右括号“)”，依次弹出栈中运算符并输出，直到左括号“(”弹出为止



考点5 栈的应用

③中缀表达式转后缀表达式（最常考）

【2024】与表达式 $x+y*(z-u)/v$ 等价的后缀表达式是

A. $xyz u - * v / +$

B. $xyz u - v / * +$

C. $+x / * y - zuv$

D. $+x * y / - zuv$



考点5 栈的应用

④递归

执行fact (3)

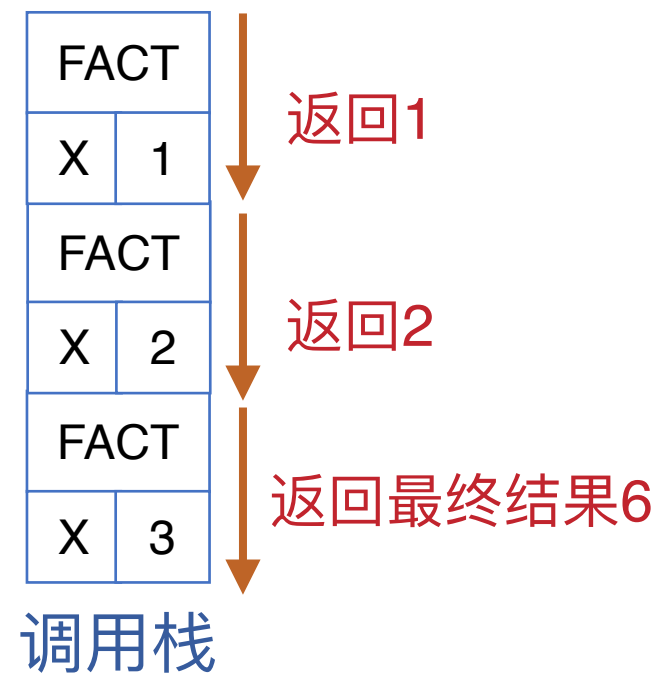
```
→ int fact(3){  
    if(x == 1)  
        return 1;  
    else return x * 2 返回最终结果3*2=6  
}
```

... 执行fact (2)

```
else return int fact(2){  
    if(x == 1)  
        return 1;  
    else return x * 1 返回2*1=2  
}
```

执行fact (1)

```
→ int fact(1){  
    if(x == 1)  
        return 1; 返回1  
    else return x * fact(x-1);  
}
```





考点5 栈的应用

④递归

【2015】 已知程序如下：

```
int S(int n)
{ return (n<=0) ? 0 : S(n-1)+n; }
void main()
{ cout << S(1); }
```

程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是

A. main()->S(1)->S(0)

B. S(0)->S(1)->main()

C. main()->S(0)->S(1)

D. S(1)->S(0)->main()



考点5 栈的应用

习题训练

14. 若 $F(n)$ 定义为

```
int F(int n){  
    if(n <= 2) return 1;  
    else return F(n-2) + F(n-3) + 1;  
}
```

请分析 $F(6)$ 的调用过程



习题训练

15.请分析func(5)的递归栈变化

```
int func(int x){  
    if(x <= 3) return 2;  
    else return func(x-2) + func(x-4);  
}
```

习题训练

x	5
FACT	

15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
    if(x <= 3) return 2;
    else return func(x-2) + func(x-4);
}
```


习题训练

x	3
FACT	
x	5
FACT	

15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
    if(x <= 3) return 2;
    else return func(x-2) + func(x-4); 执行func(3) + func(1)
}
```

↓

```
int func(int x){ 执行func(3)
    if(x <= 3) return 2;
    else return func(x-2) + func(x-4);
}
```

↘

```
int func(int x){
    if(x <= 3) return 2;
    else return func(x-2) + func(x-4);
}
```

考点5 栈的应用

选择题考频：4/16，分值：2

大题考频：0/16

习题训练

返回2

x	3
FACT	
x	5
FACT	

15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
  if(x <= 3) return 2;
  else return 2 + func(x-4);
}
```

执行func(3) + func(1)

```
int func(int x){ 执行func(3)
  if(x <= 3) return 2; 返回2
  else return func(x-2) + func(x-4);
}
```

```
int func(int x){
  if(x <= 3) return 2;
  else return func(x-2) + func(x-4);
}
```

习题训练

x	1
FACT	
x	5
FACT	

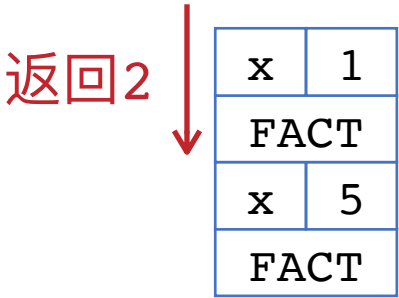
15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
    if(x <= 3) return 2;
    else return 2 + func(x-4); 执行func(3) + func(1)
}
```



```
int func(int x){ 执行func(1)
    if(x <= 3) return 2;
    else return func(x-2) + func(x-4);
}
```

习题训练

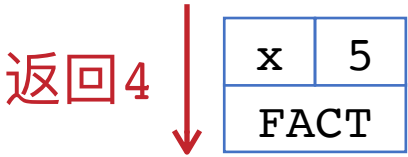


15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
    if(x <= 3) return 2;
    else return 2 + 2 ; 执行func(3) + func(1)
}
```

```
int func(int x){ 执行func(1)
    if(x <= 3) return 2; 返回2
    else return func(x-2) + func(x-4);
}
```

习题训练



15.请分析func(5)的递归栈变化

```
int func(int x){ 执行func(5)
    if(x <= 3) return 2;
    else return 2 + 2 ;
}
```

执行func(3) + func(1)
返回4



考点5 栈的应用

习题训练

16. 将中缀表达式 $A + B * (C - D)$ 转换为后缀表达式的过程中，使用一个运算符栈，扫描至 $-$ （减号）并完成这一步操作时，栈中存放的内容是（ ）（栈顶在右）

A. $+$

B. $+, *$

C. $+, *, ($

D. $+, *, (, -$



考点5 栈的应用

习题训练

17. 栈初始为空，将中缀表达式 $b - (a * c + d / e)$ 转化为等价的后缀表达式，则运算符栈中元素最多时的个数是？

A. 2

B. 3

C. 4

D. 5



习题训练

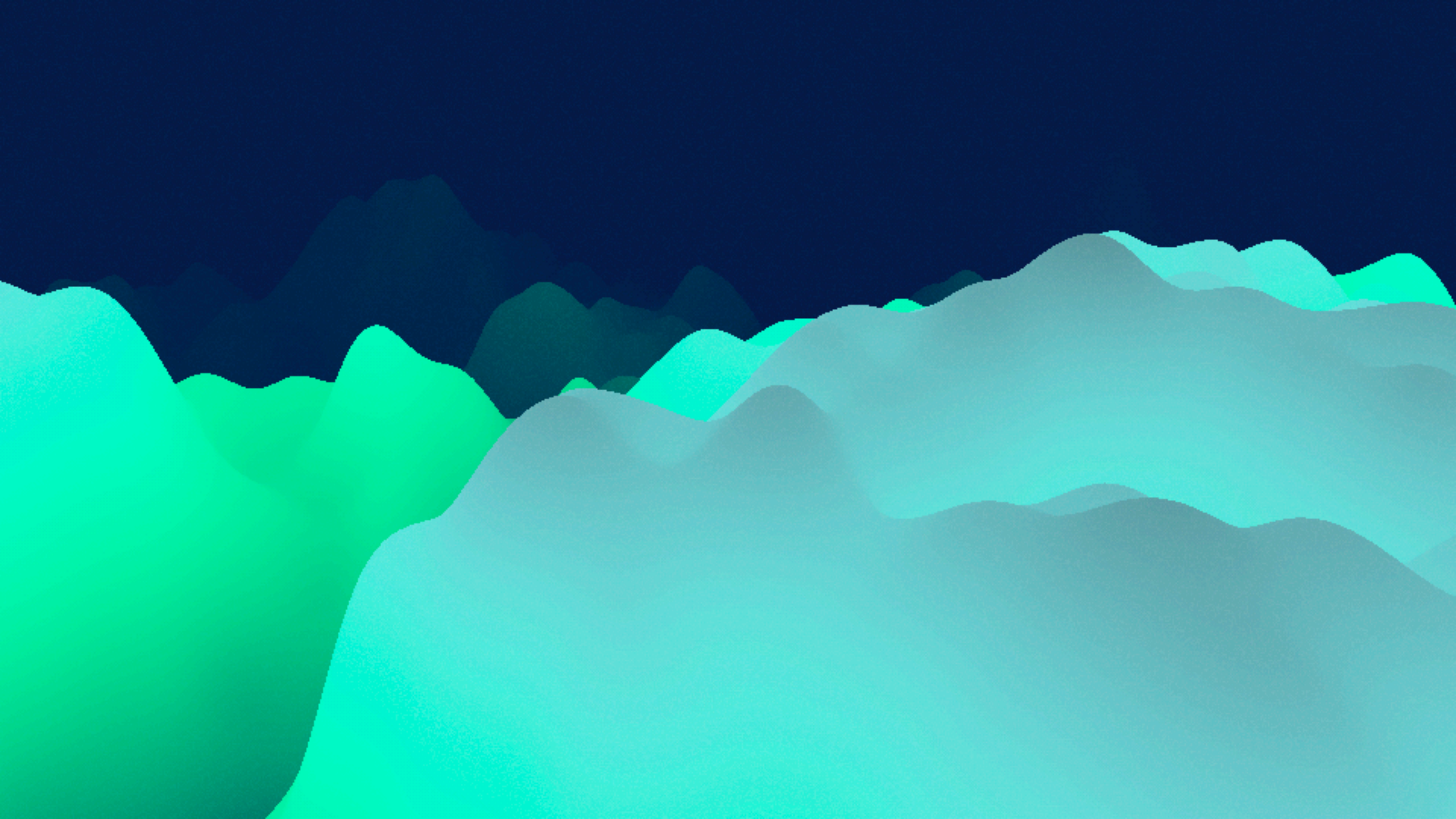
18. 若使用大小为3的操作数栈求解表达式，则下列表达式中会发生溢出的是

A. $A-B*(C-D)+E$

B. $(A-B)*C-D+E$

C. $(A-B*C+E)-D$

D. $(A-B)*(C-D)$





考点6 队列的应用

利用队列辅助实现的算法：二叉树的层次遍历、BFS、BFS求单源最短路径

队列的常用场景：

1. 要求公平调度：就绪队列、阻塞队列、各种请求队列等
2. 需要排队处理数据的缓冲区
3. 多级反馈队列调度算法



考点6 队列的应用

【2009】为解决计算机主机和打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是

- A. 栈
- B. 队列**
- C. 树
- D. 图

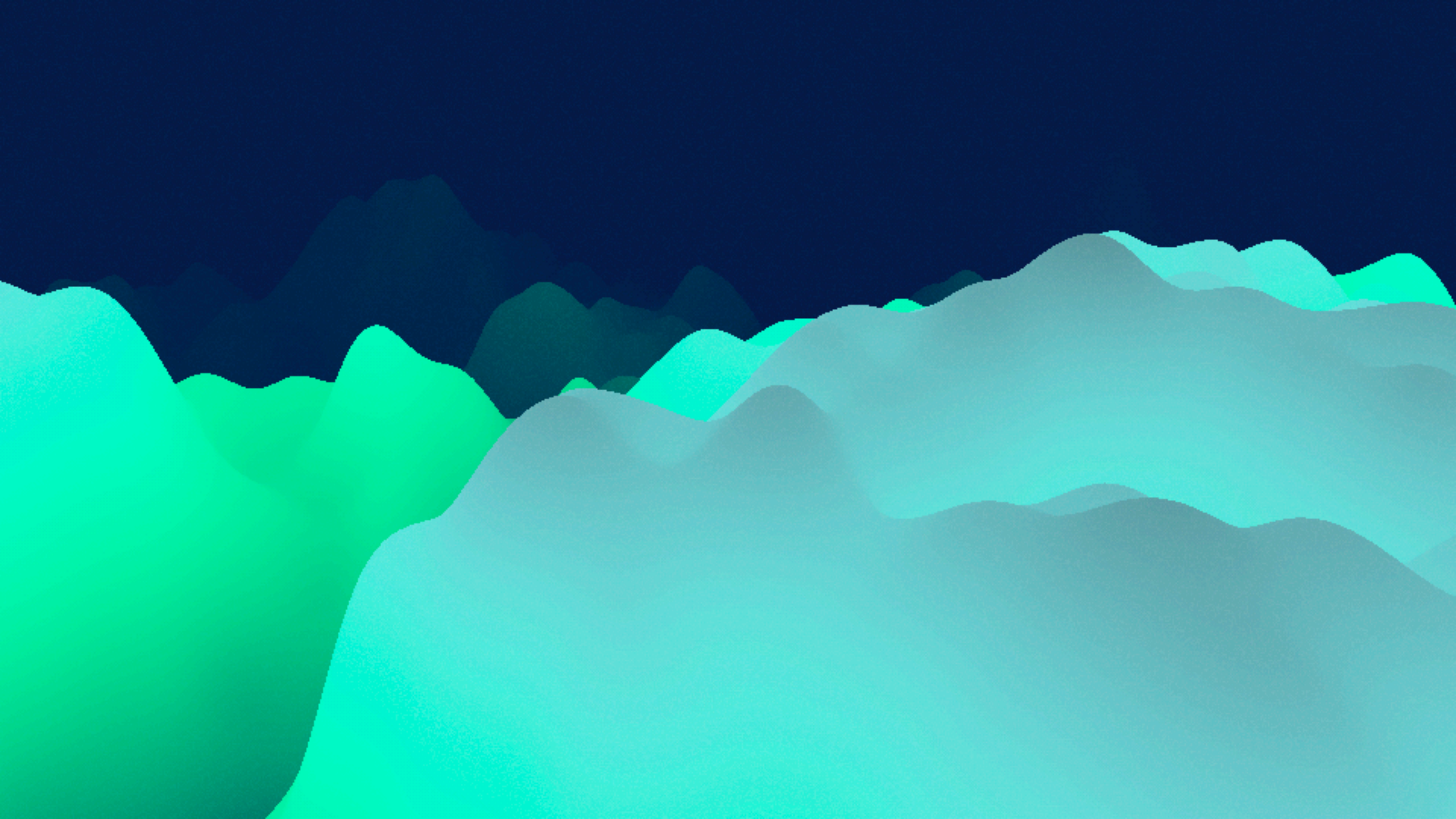


考点6 队列的应用

习题训练

19. 下列场景中，不适合使用队列的是

- A. 操作系统中的就绪进程调度
- B. 计算表达式时处理括号匹配**
- C. 网络数据包的缓冲区
- D. 图的广度优先搜索遍历



考点7 特殊矩阵

①稀疏矩阵的存储

三元组存储矩阵

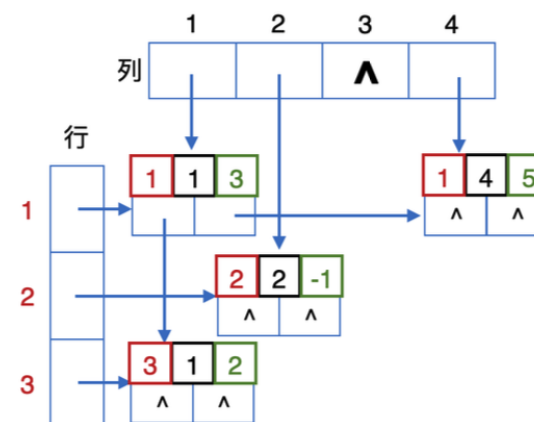
3	0	0	5
0	-1	0	0
2	0	0	0

i	j	data
0	0	3
0	3	5
1	1	-1
2	0	2

除三元组表外还需保存稀疏矩阵的行数、列数
和非零元素的个数（可选）

十字链表存储矩阵

3	0	0	5
0	-1	0	0
2	0	0	0

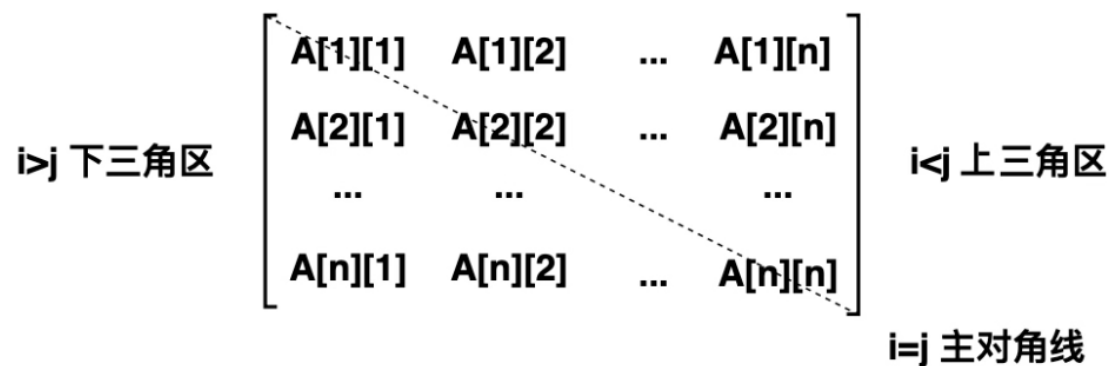




考点7 特殊矩阵

②特殊矩阵的存储

对称矩阵



$$A[i][j] = A[j][i]$$

存储时按行/列优先只存储上三角或下三角（包括对角线）



考点7 特殊矩阵

②特殊矩阵的存储

三角矩阵

$$\begin{bmatrix} A[1][1] & & & & 0 \text{或} C \\ A[2][1] & A[2][2] & & & \\ \dots & \dots & \dots & & \\ A[n][1] & A[n][2] & \dots & & A[n][n] \end{bmatrix}$$

下三角矩阵

$$\begin{bmatrix} A[1][1] & A[1][2] & \dots & A[1][n] \\ & A[2][2] & \dots & A[2][n] \\ & & \dots & \dots \\ 0 \text{或} C & & & A[n][n] \end{bmatrix}$$

上三角矩阵

除三角区域外的元素均为常数c或0

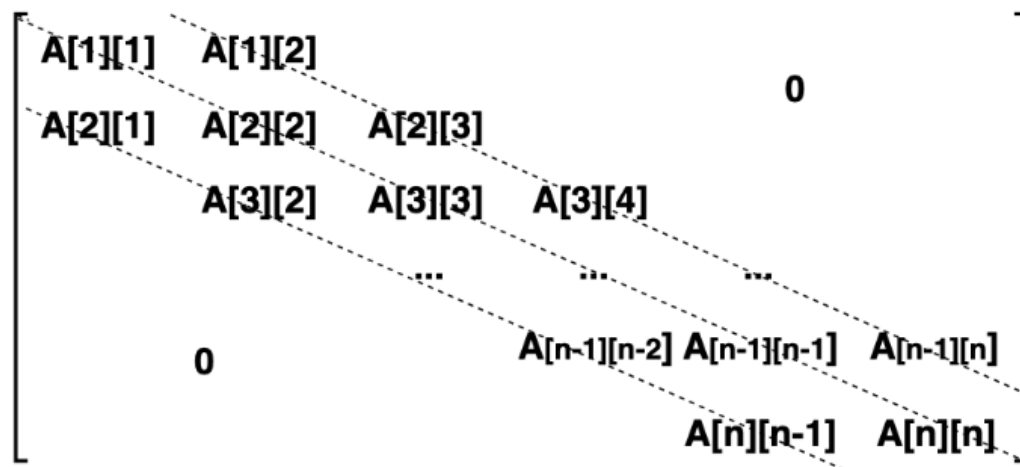
存储三角区域元素及一个常数c（c≠0时）



考点7 特殊矩阵

②特殊矩阵的存储

(三) 对角矩阵



所有的非零元素都集中在以对角线为中心的带状区域中
除了第1行和最后1行有2个元素，其余行全部都是3个元素

按行/列优先存储三条对角线上的元素

考点7 特殊矩阵

②特殊矩阵的存储

Q：元素 $A[i][j]$ 在一维数组的下标是？

一维数组下标从0开始	一维数组下标从1开始
$A[i][j]$前 有几个元素 $A[i][j]$ 对应的数组下标就是几	$A[i][j]$是 第几个元素 $A[i][j]$ 对应的数组下标就是几
$A[i][j]$ 的行列号可以从0开始，也可以从1开始	

考点7 特殊矩阵

【2016】有一个100阶的三对角矩阵M，其元素 $m_{i,j}$ ($1 \leq i, j \leq 100$) 按行优先依次压缩存入下标从0开始的一维数组N中。元素 $m_{30,30}$ 在N中的下标是

A. 86

B. 87

C. 88

D. 89

考点7 特殊矩阵

【2020】 将一个 10×10 阶对称矩阵M的上三角部分的元素 $m_{i,j}$ ($1 \leq i \leq j \leq 10$) 按列优先存入C语言的一维数组N中，元素 $m_{7,2}$ 在N中的下标是

A. 15

B. 16

C. 22

D. 23



考点7 特殊矩阵

【2021】二维数组A按行优先方式存储，每个元素占用1个存储单元。若元素A[0][0]的存储地址是100，A[3][3]的存储地址是220，则元素A[5][5]的存储地址是

A. 295

B. 300

C. 301

D. 306



考点7 特殊矩阵

【2023】若采用三元组表存储结构存储稀疏矩阵M，则除三元组表外，下列数据中还需要保存的是

I. M的行数

II. M中包含非零元素的行数

III. M的列数

IV. M中包含非零元素的列数

A. 仅I、III

B. 仅I、IV

C. 仅II、IV

D. I、II、III、IV



习题训练

20. 有一个 $n \times n$ 的对称矩阵A，将其上三角部分（含主对角线）按列存放在一维数组B中，且规定A[0][0]放在B[0]处。若 $i \leq j$ ，则A[i][j]在数组B中的下标为

A. $\frac{j(j+1)}{2} + i$

B. $\frac{i(i+1)}{2} + j$

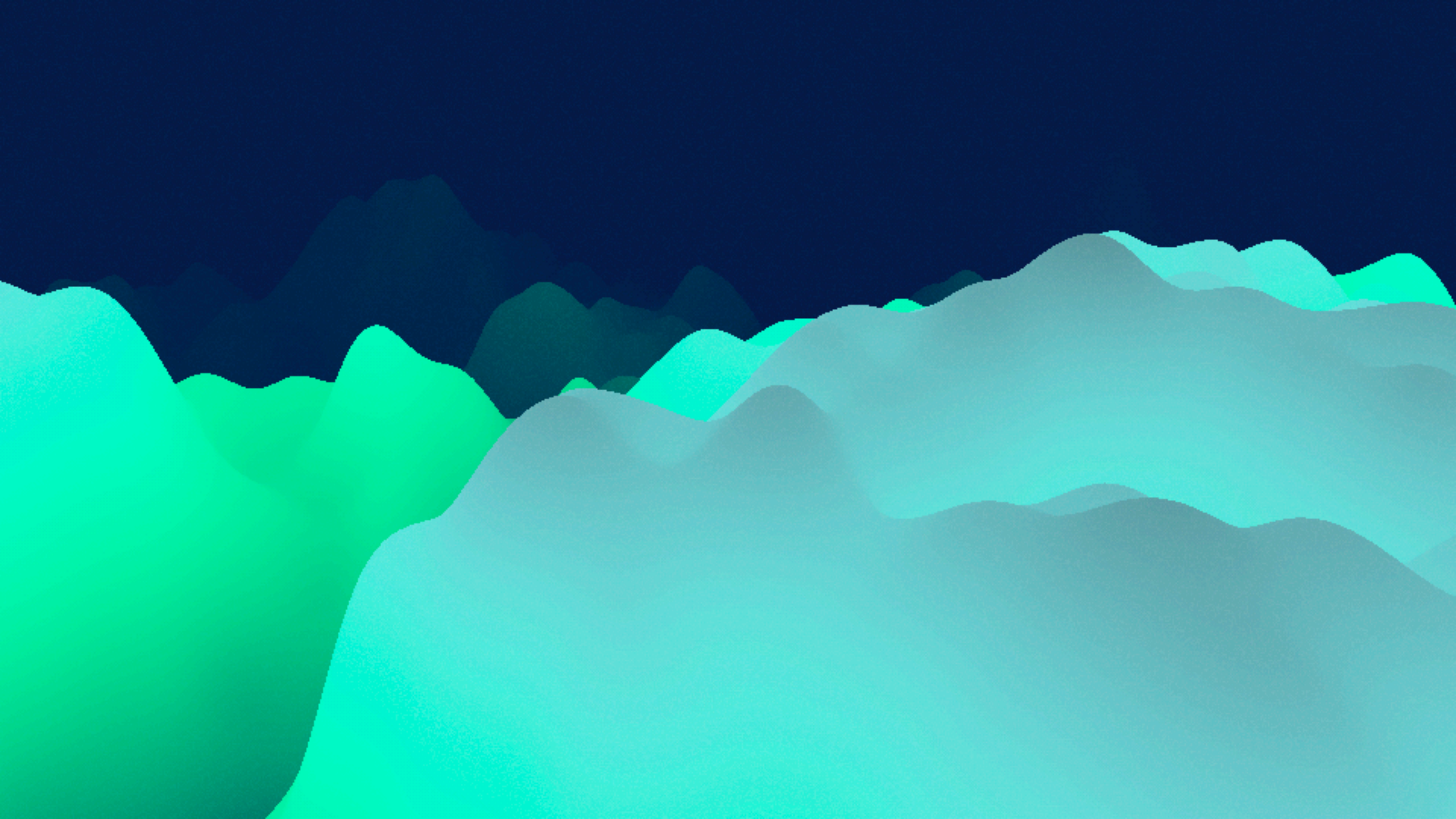
C. $\frac{n(n-1)}{2} - \frac{(n-j)(n-j-1)}{2} + i$

D. $\frac{j(j-1)}{2} + i$

习题训练

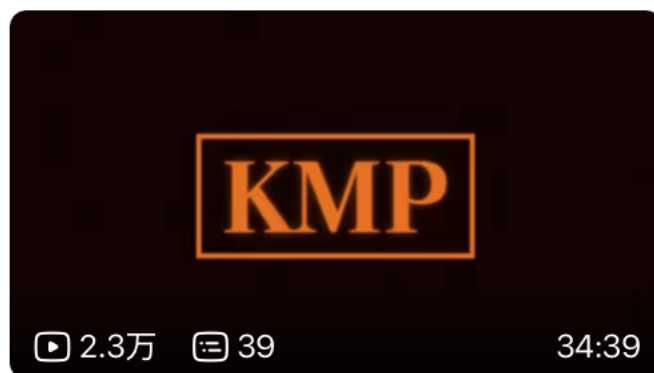
21. 一个 8×8 的三对角矩阵A，行号和列号都从0开始，将其元素按行主序压缩存入一维数组B中，每个元素占4个字节，矩阵从首地址SA连续开始存储，则元素A[5][5]的起始地址为

- A. SA+52
- B. SA+60
- C. SA+84
- D. SA+96





考点8 KMP中的next数组和nextval数组



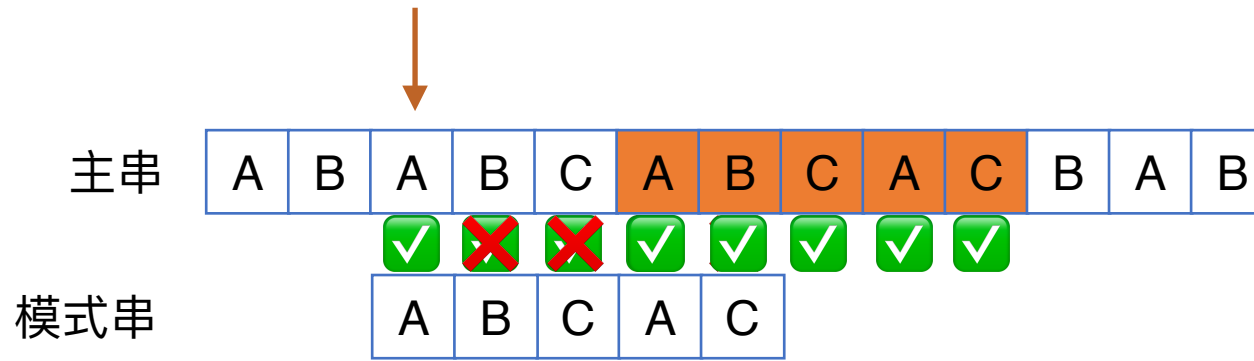
学不会KMP算法，这是个好事儿啊

2024-02-27

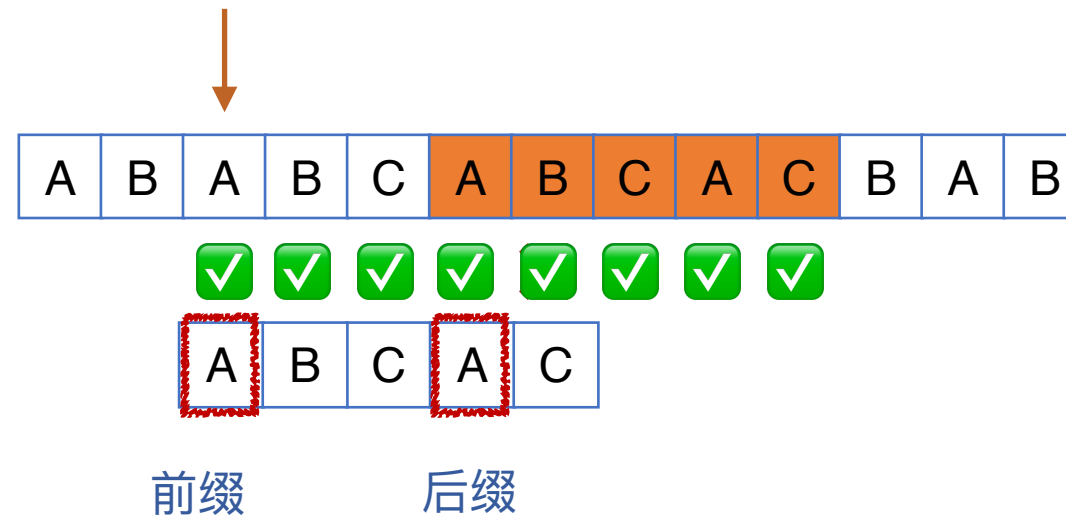
KMP本身并不难，且有概率出现在应用题中，不建议跳过！

如果觉得原理实在难以理解，也请务必掌握两个数组的求法

考点8 KMP中的next数组和nextval数组



依据ABCA的最长相等前后缀移动
前缀移动到原本后缀的位置



考点8 KMP中的next数组和nextval数组

位序	1	2	3	4	5
模式串	A	B	C	A	C
next					

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next					

假设next数组从-1开始
(将之前的next数组值全部-1)

默认情况下，next数组从0开始，前两位固定为0和1
从第三个字符开始，**next数组值=该字符之前子串的最长相等前后缀长度+1**

注意：

- 1.单个字符本身不能作为前后缀
- 2.前缀和后缀均为从左到右读

考点8 KMP中的next数组和nextval数组

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	0	1	1	1	2

假设next数组从0开始

位序	1	2	3	4	5
模式串	A	B	C	A	C
next	-1	0	0	0	1

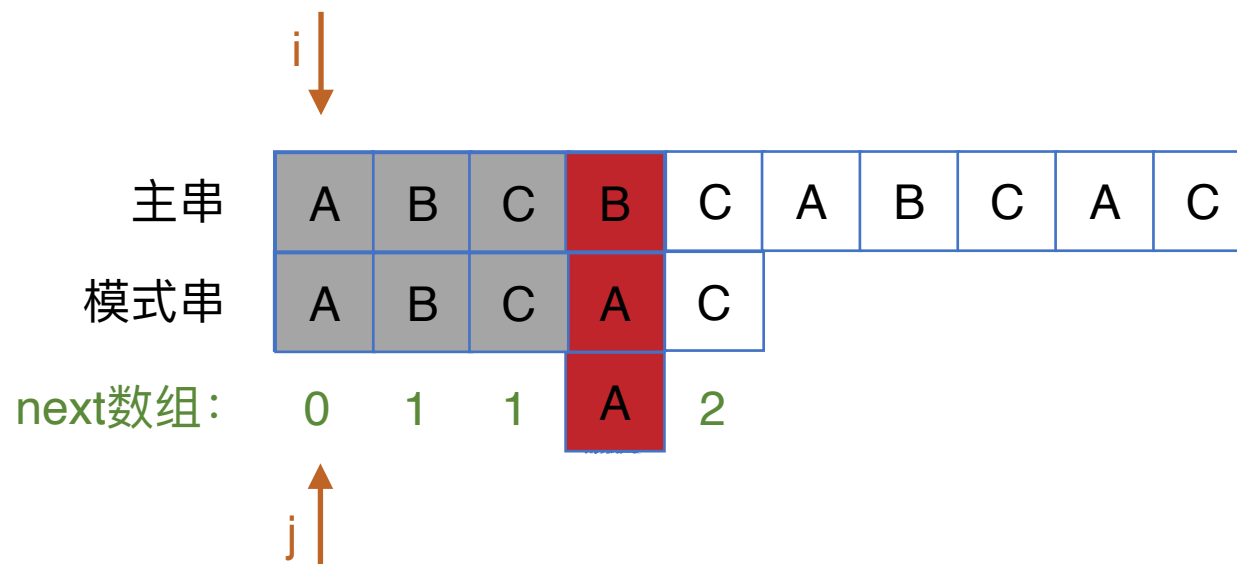
假设next数组从-1开始
(将之前的next数组值全部-1)

当模式串数组下标为*i*的位发生失配时
模式串数组第next[i]个位重新与主串当前位置进行比较

当模式串数组下标为*i*的位发生失配时
模式串数组下标为next[i]的位重新与主串当前位置进行比较



考点8 KMP中的next数组和nextval数组



主串的B与模式串的A比较
因为主串B和模式串的A并不相同
因此需要让模式串“前进”后再次比较

然而模式串“前进”后，仍然是一个A和主串的B进行比较
必定会再次比较出不相符的结果
所以这一次比较是没有意义的



考点8 KMP中的next数组和nextval数组

nextval数组的求法：

- 依次观察next数组每一位。
- 对比【本位字符】和【本位字符的next数组值对应的字符】。
- 如果二者相同，则nextval值改为next数组值对应的字符的nextval数组值。
- 如果二者不同，nextval值和本位的next数组值相同。

数组位序	1	2	3	4	5
	A	B	C	A	C

next
nextval



考点8 KMP中的next数组和nextval数组

【2015】已知字符串s为'abaabaabacacaabaabcc'，模式串t为'abaabc'。采用KMP算法进行匹配，第一次出现“失配” ($s[i] \neq s[j]$) 时， $i=j=5$ ，则下次开始匹配时，i和j的值分别为

A. $i=1, j=0$

B. $i=5, j=0$

C. $i=5, j=2$

D. $i=6, j=2$



考点8 KMP中的next数组和nextval数组

【2024】KMP算法使用修正后的next数组进行模式匹配，模式串S为'aabaab'，当主串的某个字符与s的某个字符失配时，s向右滑动的最长距离是

A. 5

B. 4

C. 3

D. 2



考点8 KMP中的next数组和nextval数组

习题训练

22. 字符串 'ababaaba' 的next数组为

A. 0, 1, 2, 3, 4, 5, 6, 7

B. 0, 1, 1, 2, 3, 4, 2, 3

C. 0, 0, 1, 2, 3, 2, 3, 2

D. 0, 1, 1, 2, 1, 1, 1, 2



考点8 KMP中的next数组和nextval数组

习题训练

23. 设模式串为 'abacabac'，其KMP的next数组为

A. -1, 0, 1, 0, 1, 2, 3, 2

B. -1, 0, 0, 1, 0, 1, 2, 3

C. -1, 0, 1, 2, 0, 1, 2, 3

D. -1, 0, 1, 2, 3, 0, 1, 2



考点8 KMP中的next数组和nextval数组

习题训练

24. 已知模式串= "ababaa", 则其nextval数组为

A. -1, 0, 1, 2, 3, 1

B. -1, 0, -1, 0, -1, 3

C. -1, 0, 1, 2, 3, 0

D. -1, 0, 1, 2, 3, 4



考点8 KMP中的next数组和nextval数组

习题训练

25. 已知主串= “aabbaabaaac”，模式串P “aabaa”，使用KMP算法（采用nextval数组），在整个匹配过程中主串和模式串进行的字符比较次数为

A. 6

B. 7

C. 8

D. 9



祝你今天特别特别开心^•u-^